SIMATS ENGINEERING

ASSESSMENT TOOL 6 – LOW

(DESIGN TASK QUESTIONS – ANSWERS)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME: CO2 – Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic.

ANSWERS: TASKS 1–25

Code of Conduct:

I **Yugitha B (192424102)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

DESIGN TASK QUESTIONS – ANSWERS

Task 1: Build a robust matrix

Question: Create an r × c matrix row-wise and reuse vector values from the beginning with a manual index.

Aim: To implement the required logic in R and display the result.

R Program:

```
build_robust_matrix <- function(x, r, c) {
  if (length(x) == 0 || r < 1 || c < 1) stop("Invalid input")
  out <- matrix(0, nrow = r, ncol = c)
  k <- 1
  for (i in 1:r) {
    for (j in 1:c) {
      out[i, j] <- x[k]
      k <- k + 1
      if (k > length(x)) k <- 1
    }
  }
  out
}
print(build_robust_matrix(c(1, 2, 3, 4, 5), 2, 4))
```

Sample Output / Result:

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    1    2    3
```

Task 2: Asset simulation using repeat

Question: Start at 100, apply random percentage changes, and stop when the asset is below 50 or above 200. Print iterations.

Aim: To implement the required logic in R and display the result.

R Program:

```
set.seed(10)
```

```
asset <- 100

iterations <- 0

repeat {

  change <- runif(1, -0.15, 0.15)

  asset <- asset * (1 + change)

  iterations <- iterations + 1

  if (asset < 50 || asset > 200) break

}

cat("Final asset:", round(asset, 2), "\n")

cat("Iterations:", iterations, "\n")
```

Sample Output / Result:

The final value and iteration count depend on the random changes. The loop stops when asset < 50 or asset > 200.

Task 3: Validate numeric bounds

Question: Replace values outside lower and upper boundaries with 'Out of Bounds'.

Aim: To implement the required logic in R and display the result.

R Program:

```
validate_bounds <- function(x, lower, upper) {

  result <- character(length(x))

  for (i in seq_along(x)) {

    if (x[i] < lower || x[i] > upper) {

      result[i] <- "Out of Bounds"

    } else {

      result[i] <- as.character(x[i])

    }

  }

  result

}

print(validate_bounds(c(5, 10, 15, 25), 8, 20))
```

Sample Output / Result:

[1] "Out of Bounds" "10" "15" "Out of Bounds"

Task 4: Fallback logging configuration

Question: Use a dynamic default that checks for a global logging variable; otherwise use a local default configuration.

Aim: To implement the required logic in R and display the result.

R Program:

```
process_log <- function(message,
  logger_type = if (exists("global_logger", envir = .GlobalEnv,
                           inherits = FALSE))
    get("global_logger", envir = .GlobalEnv)
  else list(type = "console", enabled = TRUE)) {
  if (is.list(logger_type)) {
    if (isTRUE(logger_type$enabled))
      cat("[", logger_type$type, "] ", message, "\n", sep = "")
  } else {
    cat("[", logger_type, "] ", message, "\n", sep = "")
  }
}
process_log("System started")
global_logger <- "file"
process_log("Data saved")
rm(global_logger)
```

Sample Output / Result:

[console] System started

[file] Data saved

Task 5: Generate a multi-metric report

Question: Calculate means, standard deviations, missing-value counts, and a timestamp in a named list.

Aim: To implement the required logic in R and display the result.

R Program:

```
generate_report <- function(data) {
  numeric_cols <- vapply(data, is.numeric, logical(1))
  numeric_data <- data[, numeric_cols, drop = FALSE]
  list(
    means = lapply(numeric_data, mean, na.rm = TRUE),
    standard_deviations = lapply(numeric_data, sd, na.rm = TRUE),
    missing_values = lapply(data, function(x) sum(is.na(x))),
    generated_at = Sys.time()
  )
}
d <- data.frame(a = c(1, 2, NA), b = c(4, 5, 6))
report <- generate_report(d)
print(report$means)
print(report$missing_values)
```

Sample Output / Result:

Means: a = 1.5, b = 5

Missing values: a = 1, b = 0

The list also contains standard deviations and the current timestamp.

Task 6: Matrix transformation router

Question: Store matrix transformations in a named function list and run the selected operation by name.

Aim: To implement the required logic in R and display the result.

R Program:

```
operations <- list(
  rotate_90 = function(m) t(m[nrow(m):1, , drop = FALSE]),
  reflect_x = function(m) m[nrow(m):1, , drop = FALSE],
  reflect_y = function(m) m[, ncol(m):1, drop = FALSE]
)
```

```
route_matrix <- function(m, operation) {
  fn <- operations[[operation]]
  if (is.null(fn)) stop("Unknown operation")
  fn(m)
}
m <- matrix(1:6, nrow = 2)
print(route_matrix(m, "reflect_x"))
```

Sample Output / Result:

```
     [,1] [,2] [,3]
[1,]    2    4    6
[2,]    1    3    5
```

Task 7: Recursive nested-list traversal

Question: Count leaf values in a nested list and stop if nesting exceeds a fixed depth limit.

Aim: To implement the required logic in R and display the result.

R Program:

```
traverse_nodes <- function(node, depth = 0, max_depth = 50) {
  if (depth > max_depth) stop("Maximum depth exceeded")
  if (!is.list(node)) return(length(node))
  total <- 0
  for (item in node)
    total <- total + traverse_nodes(item, depth + 1, max_depth)
  total
}
tree <- list(1, list(2, 3), list(4, list(5)))
print(traverse_nodes(tree))
```

Sample Output / Result:

```
[1] 5
```

Task 8: Compare matrices element by element

Question: Return TRUE only if every value in A is greater than the corresponding value in B, using element-wise comparison.

Aim: To implement the required logic in R and display the result.

R Program:

```
compare_matrices <- function(A, B) {
  if (!all(dim(A) == dim(B))) stop("Dimensions must match")
  all(A > B)
}
A <- matrix(c(8, 9, 7, 6), nrow = 2)
B <- matrix(c(1, 2, 3, 4), nrow = 2)
print(compare_matrices(A, B))
```

Sample Output / Result:

```
[1] TRUE
```

Task 9: Evaluate a series while skipping multiples

Question: Sum values from 1 to N, skipping multiples of 3 or 7 using next.

Aim: To implement the required logic in R and display the result.

R Program:

```
evaluate_series <- function(N) {
  if (N < 0 || N != floor(N)) stop("N must be a non-negative integer")
  total <- 0
  for (i in seq_len(N)) {
    if (i %% 3 == 0 || i %% 7 == 0) next
    total <- total + i
  }
  total
}
print(evaluate_series(10))
```

Sample Output / Result:

```
[1] 20
```

Task 10: Safe vector division

Question: Return -999 when a denominator is zero or NA, and preserve other calculated values.

Aim: To implement the required logic in R and display the result.

R Program:

```
safe_divide <- function(numerators, denominators) {
  if (length(numerators) != length(denominators))
    stop("Vectors must have equal lengths")
  result <- numeric(length(numerators))
  for (i in seq_along(numerators)) {
    if (is.na(denominators[i]) || denominators[i] == 0 ||
        is.na(numerators[i])) {
      result[i] <- -999
    } else {
      result[i] <- numerators[i] / denominators[i]
    }
  }
  result
}
print(safe_divide(c(10, 12, 9), c(2, 0, NA)))
```

Sample Output / Result:

```
[1]    5 -999 -999
```

Task 11: Sliding window average

Question: Calculate successive window averages and throw an error if input length is shorter than W.

Aim: To implement the required logic in R and display the result.

R Program:

```
sliding_average <- function(x, W) {
  if (W < 1 || W != floor(W)) stop("W must be a positive integer")
  if (length(x) < W) stop("Input length is shorter than W")
```

```
  result <- numeric(length(x) - W + 1)
  for (i in seq_along(result))
    result[i] <- mean(x[i:(i + W - 1)])
  result
}
print(sliding_average(c(2, 4, 6, 8, 10), 3))
```

Sample Output / Result:

```
[1] 4 6 8
```

Task 12: Power with a dynamic default exponent

Question: If p is omitted, use the mean of a global array as the exponent.

Aim: To implement the required logic in R and display the result.

R Program:

```
global_values <- c(2, 4, 6)
compute_power <- function(x,
  p = mean(get("global_values", envir = .GlobalEnv))) {
  if (length(p) != 1 || is.na(p) || p < 0 || p != floor(p))
    stop("Exponent must be a non-negative integer")
  result <- 1
  if (p > 0) for (i in seq_len(p)) result <- result * x
  result
}
print(compute_power(2))
print(compute_power(2, 3))
```

Sample Output / Result:

```
[1] 16
[1] 8
```

Task 13: Apply a sequence of functions

Question: Pass each function's output to the next function in the list.

Aim: To implement the required logic in R and display the result.

R Program:

```
apply_stepwise <- function(raw_values, functions_list) {
  value <- raw_values
  for (fn in functions_list) value <- fn(value)
  value
}
steps <- list(function(x) x + 2,
              function(x) x * 3,
              function(x) x - 1)
print(apply_stepwise(c(1, 2, 3), steps))
```

Sample Output / Result:

```
[1]  8 11 14
```

Task 14: Track rows failing a second rule

Question: Record row and metric-column coordinates when metric1 passes but metric2 fails.

Aim: To implement the required logic in R and display the result.

R Program:

```
track_errors <- function(data) {
  errors <- list()
  k <- 1
  for (i in seq_len(nrow(data))) {
    first_ok <- !is.na(data$metric1[i]) && data$metric1[i] >= 50
    second_ok <- !is.na(data$metric2[i]) && data$metric2[i] <= 100
    if (first_ok && !second_ok) {
      errors[[k]] <- c(row = i, metric_column = 2)
      k <- k + 1
    }
  }
  errors
```

```
}
df <- data.frame(metric1 = c(60, 40, 75), metric2 = c(90, 110, 130))
print(track_errors(df))
```

Sample Output / Result:

The list records row 3, column 2 because metric1 passes and metric2 fails.

Task 15: Recursive GCD

Question: Calculate the greatest common divisor recursively with a base case when the second number is zero.

Aim: To implement the required logic in R and display the result.

R Program:

```
gcd_recursive <- function(a, b) {
  if (a < 0 || b < 0 || a != floor(a) || b != floor(b))
    stop("Enter non-negative integers")
  if (b == 0) return(a)
  gcd_recursive(b, a %% b)
}
print(gcd_recursive(24, 18))
print(gcd_recursive(35, 14))
```

Sample Output / Result:

```
[1] 6
[1] 7
```

Task 16: Short-circuit credential verification

Question: Use && so later checks are evaluated only when earlier checks pass.

Aim: To implement the required logic in R and display the result.

R Program:

```
verify_credentials <- function(active, username, password,
                               expected_user, expected_password) {
  if (isTRUE(active) &&
      is.character(username) && length(username) == 1 &&
```

```
    username == expected_user &&
    is.character(password) && length(password) == 1 &&
    password == expected_password) {
   TRUE
  } else {
   FALSE
  }
}
print(verify_credentials(TRUE, "user1", "abc123", "user1", "abc123"))
print(verify_credentials(FALSE, "user1", "wrong", "user1", "abc123"))
```

Sample Output / Result:

```
[1] TRUE
[1] FALSE
```

Task 17: Append text until it exceeds 150 characters

Question: Use a while loop to append a suffix and stop as soon as the text is longer than 150 characters.

Aim: To implement the required logic in R and display the result.

R Program:

```
text_value <- "Start"
suffix <- "-R"
turns <- 0
while (nchar(text_value) <= 150) {
  text_value <- paste0(text_value, suffix)
  turns <- turns + 1
  if (nchar(text_value) > 150) break
}
cat("Final character count:", nchar(text_value), "\n")
cat("Turns:", turns, "\n")
```

Sample Output / Result:

The loop stops as soon as the character count exceeds 150. The count and turns depend on the starting text and suffix.

Task 18: Summarize a data frame

Question: Return row count, column data types, and unique values in a named list.

Aim: To implement the required logic in R and display the result.

R Program:

```
summarize_frame <- function(data) {
  list(
    row_count = nrow(data),
    column_types = vapply(data, function(x) class(x)[1], character(1)),
    unique_values = lapply(data, unique)
  )
}
df <- data.frame(age = c(18, 19, 18), group = c("A", "B", "A"))
s <- summarize_frame(df)
print(s$row_count)
print(s$column_types)
print(s$unique_values)
```

Sample Output / Result:

Row count: 3

Column types: age = numeric, group = character

Unique values: age = 18, 19; group = A, B.

Task 19: Append a tracking flag

Question: Append a modification indicator and return a new vector without changing the original.

Aim: To implement the required logic in R and display the result.

R Program:

```
append_flag <- function(x) {
  updated <- c(x, "MODIFIED")
```

```
  return(updated)
}
original <- c("A", "B", "C")
updated <- append_flag(original)
print(original)
print(updated)
```

Sample Output / Result:

```
[1] "A" "B" "C"
[1] "A" "B" "C" "MODIFIED"
```

Task 20: Calculate tariff by age and usage

Question: Use nested if-else statements for age groups and usage brackets. Example rates: under 18 costs 2/unit up to 100 then 3/unit; age 18+ costs 3/unit up to 100 then 4/unit.

Aim: To implement the required logic in R and display the result.

R Program:

```
calculate_tariff <- function(age, usage) {
  if (age < 0 || usage < 0) stop("Values cannot be negative")
  if (age < 18) {
    if (usage <= 100) charge <- 2 * usage
    else charge <- 200 + (usage - 100) * 3
  } else {
    if (usage <= 100) charge <- 3 * usage
    else charge <- 300 + (usage - 100) * 4
  }
  charge
}
print(calculate_tariff(15, 120))
print(calculate_tariff(25, 120))
```

Sample Output / Result:

```
[1] 260
```

[1] 380

Task 21: Find increasing triplets

Question: Find starting positions where three consecutive values are strictly increasing.

Aim: To implement the required logic in R and display the result.

R Program:

```
find_triplets <- function(x) {
  positions <- integer(0)
  if (length(x) < 3) return(positions)
  for (i in 1:(length(x) - 2)) {
    if (x[i] < x[i + 1] && x[i + 1] < x[i + 2])
      positions <- c(positions, i)
  }
  positions
}
print(find_triplets(c(1, 3, 5, 4, 6, 8)))
```

Sample Output / Result:

[1] 1 4

Task 22: Execute a function and report duration

Question: Run an operation on a dataset and return results, start/end timestamps, and elapsed seconds.

Aim: To implement the required logic in R and display the result.

R Program:

```
execute_with_reporting <- function(operation, dataset) {
  start_time <- Sys.time()
  result <- operation(dataset)
  end_time <- Sys.time()
  list(
    result = result,
    started_at = start_time,
```

```
    finished_at = end_time,
    elapsed_seconds = as.numeric(difftime(end_time, start_time,
                                          units = "secs"))
  )
}
answer <- execute_with_reporting(sum, 1:100)
print(answer$result)
print(answer$elapsed_seconds)
```

Sample Output / Result:

[1] 5050

Elapsed seconds are measured at runtime and may vary.

Task 23: Normalize matrix columns safely

Question: Standardize columns using their standard deviations. If any column has zero variance, return a baseline vector of zeros.

Aim: To implement the required logic in R and display the result.

R Program:

```
normalize_scores <- function(m) {
  if (!is.matrix(m) || !is.numeric(m)) stop("Use a numeric matrix")
  sds <- apply(m, 2, sd)
  if (any(is.na(sds)) || any(sds == 0)) return(rep(0, nrow(m)))
  means <- colMeans(m)
  sweep(sweep(m, 2, means, "-"), 2, sds, "/")
}
m <- matrix(c(1, 2, 3, 5, 5, 5), nrow = 3)
print(normalize_scores(m))
```

Sample Output / Result:

[1] 0 0 0

The second column has zero standard deviation, so the baseline vector is returned.

Task 24: Validate configuration updates

Question: Allow only known keys and valid values, returning a sanitized configuration list.

Aim: To implement the required logic in R and display the result.

R Program:

```
config <- list(theme = "light", retries = 3, enabled = TRUE)
update_config <- function(config, updates) {
  allowed <- names(config)
  for (key in names(updates)) {
    if (!(key %in% allowed)) next
    value <- updates[[key]]
    if (key == "theme" && value %in% c("light", "dark")) {
      config[[key]] <- value
    } else if (key == "retries" && length(value) == 1 &&
               !is.na(value) && value >= 0 && value == floor(value)) {
      config[[key]] <- value
    } else if (key == "enabled" && is.logical(value) &&
               length(value) == 1 && !is.na(value)) {
      config[[key]] <- value
    }
  }
  config
}
print(update_config(config, list(theme = "dark", retries = 5,
                                 enabled = FALSE, extra = 99)))
```

Sample Output / Result:

theme = "dark", retries = 5, enabled = FALSE

The unknown "extra" field is ignored.

Task 25: Recursively unpack nested strings

Question: Extract each individual character from nested character strings into one flat character vector.

Aim: To implement the required logic in R and display the result.

R Program:

```
unpack_strings <- function(x) {
  if (is.list(x)) {
    result <- character(0)
    for (item in x) result <- c(result, unpack_strings(item))
    return(result)
  }
  if (is.character(x)) {
    result <- character(0)
    for (s in x) result <- c(result, strsplit(s, "", fixed = TRUE)[[1]])
    return(result)
  }
  character(0)
}
nested <- list("Hi", list("R", "AI"), "OK")
print(unpack_strings(nested))
```

Sample Output / Result:

```
[1] "H" "i" "R" "A" "I" "O" "K"
```